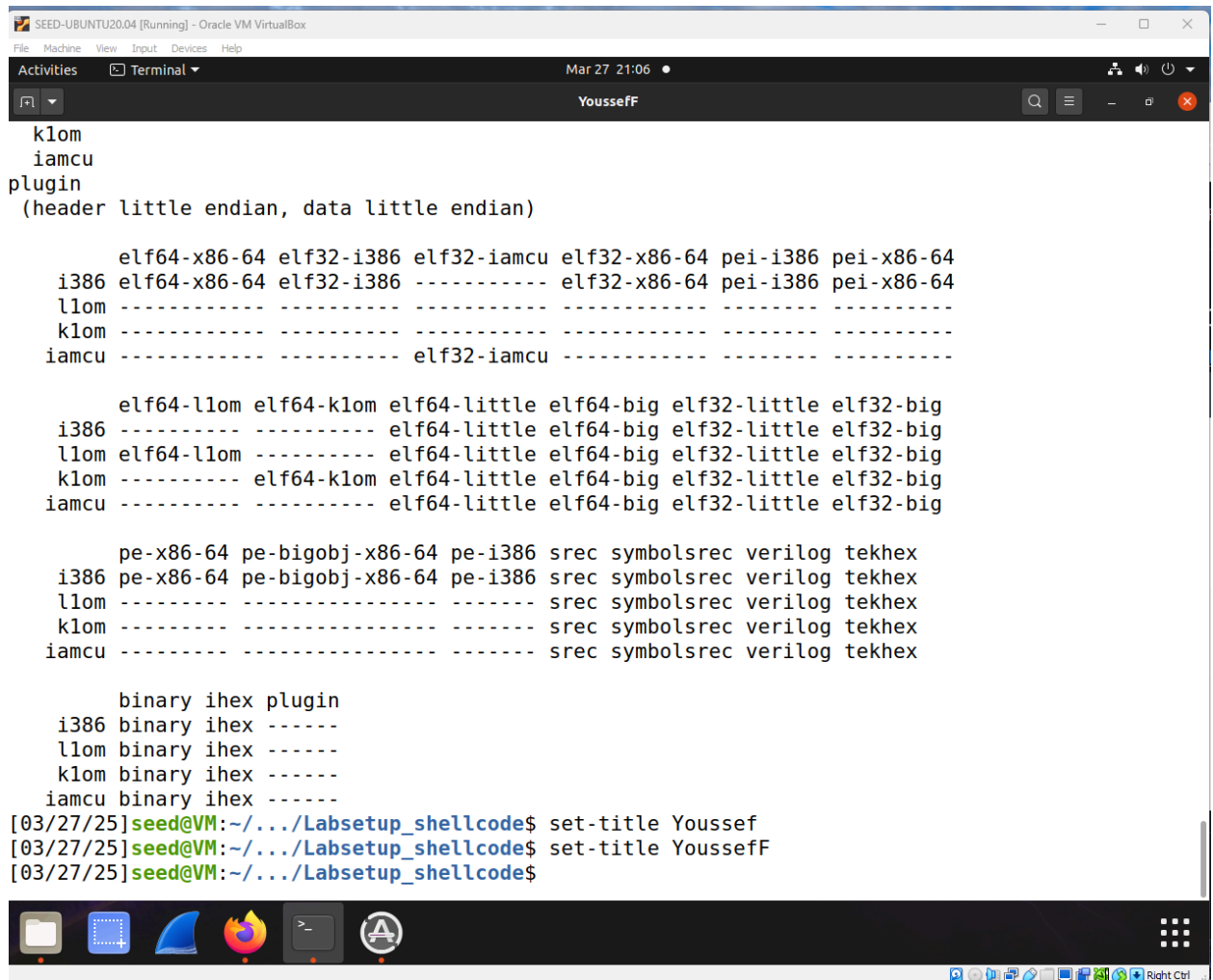


Shellcode Lab report



```
SEED-UBUNTU20.04 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Activities Terminal Mar 27 21:06
YoussefF

k1om
i1omcu
plugin
(header little endian, data little endian)

elf64-x86-64 elf32-i386 elf32-iamcu elf32-x86-64 pei-i386 pei-x86-64
i386 elf64-x86-64 elf32-i386 ----- elf32-x86-64 pei-i386 pei-x86-64
llom -----
k1om -----
iamcu ----- elf32-iamcu -----

elf64-llom elf64-k1om elf64-little elf64-big elf32-little elf32-big
i386 ----- elf64-little elf64-big elf32-little elf32-big
llom elf64-llom ----- elf64-little elf64-big elf32-little elf32-big
k1om ----- elf64-k1om elf64-little elf64-big elf32-little elf32-big
iamcu ----- elf64-little elf64-big elf32-little elf32-big

pe-x86-64 pe-bigobj-x86-64 pe-i386 srec symbolsrec verilog tekhex
i386 pe-x86-64 pe-bigobj-x86-64 pe-i386 srec symbolsrec verilog tekhex
llom ----- srec symbolsrec verilog tekhex
k1om ----- srec symbolsrec verilog tekhex
iamcu ----- srec symbolsrec verilog tekhex

binary ihex plugin
i386 binary ihex -----
llom binary ihex -----
k1om binary ihex -----
iamcu binary ihex -----
[03/27/25]seed@VM:~/.../Labsetup_shellcode$ set-title Youssef
[03/27/25]seed@VM:~/.../Labsetup_shellcode$ set-title YoussefF
[03/27/25]seed@VM:~/.../Labsetup_shellcode$
```

- 1.
2. Question 2:
No, they are not. While working on this lab, I noticed that some x86 instructions are short, like one byte, and others are longer. It depends on what the instruction is trying to do. More complex instructions use more bytes. The x86 system does not use fixed-length instructions, which means their sizes can vary.
3. Question 3:
When I looked at the disassembled version of my code, I noticed that some of the instructions appeared slightly different from what I originally wrote. It also looked like the order of some bytes was flipped. I learned that this is because the system uses a

little-endian format. In little-endian, the computer stores the smallest byte of a number first. This affects how the instructions and data appear in memory. Even though the bytes look out of order, they still represent the same values when the system reads them.

```
[03/27/25]seed@VM:~/.../Labsetup_shellcode$ nasm -f elf32 mysh2.s -o mysh2.o
[03/27/25]seed@VM:~/.../Labsetup_shellcode$ ld -m elf_i386 mysh2.o -o mysh2
[03/27/25]seed@VM:~/.../Labsetup_shellcode$ ./mysh2
Segmentation fault
[03/27/25]seed@VM:~/.../Labsetup_shellcode$ █
```

4.

When I first ran the second version of the shellcode (mysh2), it did not work. I found out this was because the memory section where the shellcode is placed is not allowed to run code by default. To fix it, I used the `--omagic` option when linking the file. This changed the memory settings so that the code could be executed. After that, I reran the shellcode and it worked as expected, opening a shell.

```
SEED-UBUNTU20.04 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Activities Terminal Mar 27 21:12 Yousseff

elf64-x86-64 elf32-i386 elf32-iamcu elf32-x86-64 pei-i386 pei-x86-64
i386 elf64-x86-64 elf32-i386 ----- elf32-x86-64 pei-i386 pei-x86-64
llom -----
klom -----
iamcu ----- elf32-iamcu -----

elf64-llom elf64-klom elf64-little elf64-big elf32-little elf32-big
i386 ----- elf64-little elf64-big elf32-little elf32-big
llom elf64-llom ----- elf64-little elf64-big elf32-little elf32-big
klom ----- elf64-klom elf64-little elf64-big elf32-little elf32-big
iamcu ----- elf64-little elf64-big elf32-little elf32-big

pe-x86-64 pe-bigobj-x86-64 pe-i386 srec symbolsrec verilog tekhex
i386 pe-x86-64 pe-bigobj-x86-64 pe-i386 srec symbolsrec verilog tekhex
llom ----- srec symbolsrec verilog tekhex
klom ----- srec symbolsrec verilog tekhex
iamcu ----- srec symbolsrec verilog tekhex

binary ihex plugin
i386 binary ihex -----
llom binary ihex -----
klom binary ihex -----
iamcu binary ihex -----
[03/27/25]seed@VM:~/.../Labsetup_shellcode$ ld mysh_64.o -o mysh_64
[03/27/25]seed@VM:~/.../Labsetup_shellcode$ ./mysh_64
$ echo ${0}
/bin//sh
$ echo $$
4360
$ █
```

5.